“Multi-User Chat Application Using Linux System Calls”

A Project Report Submitted in partial fulfillment of the requirements for the course

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Yashwanth	**2520030259**
Bharath	**2520030169**
Praneeth	**2520030186**

Under the Esteemed Guidance of

Dr. Soumya

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



DECLARATION

The project report entitled "**Multi-User Chat Application Using Linux System Calls**" is a record of bona fide work carried out by **Yashwanth – 2520030259, Bharath – 2520030169, and Praneeth – 2520030186**, submitted in partial fulfillment of the requirements for the Operating Systems and System Programming (OSSP) course at KLH University. The project demonstrates Linux system calls and operating-system concepts through a multi-user client-server chat application.The work presented in this report has been prepared by our team for academic purposes. Any external technical references used are acknowledged in the References section.

Yashwanth 2520030259

Bharath 2520030169

Praneeth 2520030186

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini-project report entitled "**Multi-User Chat Application Using Linux System Calls**" is a bona fide work carried out by **Yashwanth – 2520030259, Bharath – 2520030169, and Praneeth – 2520030186,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Yashwanth 2520030259

Bharath 2520030169

Praneeth 2520030186

ABSTRACT

The Multi-User Chat Application Using Linux System Calls is a terminal-based client-server application developed in C for Ubuntu/Linux and WSL. It allows multiple users to connect to a central server over TCP and exchange messages. The project demonstrates core operating-system concepts through practical implementation.

The server creates a listening TCP socket, accepts client connections, and uses fork() to create a child process for each client. Child processes report client events to the parent process through a pipe. The parent process maintains the active-client list and performs broadcast messaging, private-message routing, online-user listing, and client cleanup. The application also uses signal handling to support orderly shutdown and to reap terminated child processes.

The client uses select() to monitor both keyboard input and incoming server data. Users can send broadcast messages and use /help, /users, /msg <user> <message>, /broadcast <message>, and /quit. The project brings together sockets, processes, IPC, signals, file descriptors, and resource management.

Keywords: Linux, C, system calls, fork, pipe, TCP sockets, inter-process communication, signals, select, OSSP.

INDEX

S. No.	Chapter	Topics / Subtopics	Page No.
		Acknowledgement	i
		Abstract	ii
1	**Introduction**		**1–5**
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	**OS Concepts and Background Study**		**6–12**
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	**System Requirements**		**13–17**
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	**Technology and Tools Used**		**18–22**
		4.1 C Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 Linux Commands and Utilities	
		4.5 Other Tools Used	
5	**System Architecture and Design**		**23–29**
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process Implementation	
		6.6 IPC Implementation	
		6.7 Resource Management	
7	**Execution and Demonstration**		**46–50**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion and Results**	8.1 Summary of the Project	**51–53**
		8.2 Objectives Achieved	
		8.3 Results and Learning Outcomes	
9	**References**	9.1 Books	**54–56**
		9.2 Linux / GNU Documentation	
		9.3 Online Resources	

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

Communication applications allow people to exchange information over a network. A multi-user chat application is a useful system-programming project because it combines networking, concurrent execution, and operating-system resource management. In this project, multiple terminal clients connect to one server and exchange messages using TCP sockets.

1.2 Problem Statement

A basic single-user program does not demonstrate how an operating system handles multiple clients, concurrent processes, communication channels, and network resources. The project addresses this problem by implementing a multi-user chat application that uses Linux system calls to accept connections, create processes, transfer events between processes, and route messages.

1.3 Scope of the Project

The project is intended for an educational demonstration in Ubuntu/Linux or WSL. It supports multiple connected clients, username-based sessions, broadcast messages, private messages, online-user listing, help, and quitting. It is not designed as a production-grade public messaging service.

1.4 Objectives of the Project

- Create a TCP server and client using the socket API.
- Support concurrent clients using fork().
- Use pipe() for inter-process communication between child processes and the parent server.
- Maintain a central active-client list in the parent process.
- Support broadcast and private messaging, online-user listing, and help commands.
- Use signals and waitpid() for server shutdown and child-process cleanup.
- Provide reproducible compilation and execution steps.

1.5 Importance of the Application

The project provides practical experience with Linux system calls and demonstrates the relationship between processes, IPC, sockets, signals, and file descriptors. It helps students connect theoretical operating-system concepts with observable program behavior.

1.6 Target Users / Audience

The primary audience is students learning Operating Systems and System Programming, instructors evaluating Linux system-call concepts, and developers who want a small reference example of process-based client-server communication.

CHAPTER 2 — OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating-System Concepts Related to the Project

System calls provide a controlled interface between a user-space program and the kernel. This project uses system calls and POSIX APIs for networking, process creation, IPC, signals, input/output, and resource cleanup.

2.1.1 Process Creation — fork()

fork() creates a child process from the calling process. It returns zero in the child, the child process ID in the parent, and -1 on failure. The server uses a child process to handle each client connection.

2.1.2 Inter-Process Communication — pipe()

pipe() creates a unidirectional communication channel represented by file descriptors. A child writes structured events to the pipe, and the parent reads them to update the client table and route messages.

2.1.3 TCP Sockets

TCP provides a reliable, ordered byte stream. The server uses socket(), bind(), listen(), and accept(); the client uses socket() and connect(). send() and recv() transfer data over established connections.

2.1.4 Signals and Child Cleanup

SIGINT can request server shutdown when Ctrl+C is pressed. SIGCHLD indicates a child process state change. waitpid() with WNOHANG can collect terminated children without blocking normal server work.

2.2 Existing Methods / Algorithms

A simple chat system may handle only one client at a time, preventing other users from communicating while that client is being served. Another approach is to use threads or an event loop. This project uses a process-per-client model and a pipe for child-to-parent event communication.

2.3 Proposed Approach

The proposed approach separates client handling from central state management. Each child handles its connected client, while the parent owns the active-client list and routes events. This design demonstrates both process concurrency and IPC.

2.4 Advantages of the Proposed Approach

- Supports several clients concurrently.
- Demonstrates fork(), pipe(), TCP sockets, and signal handling.
- Keeps central client-state updates in the parent process.
- Separates the client and server source code.
- Can be built and tested using common Linux tools.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 Hardware Requirements

A desktop or laptop capable of running Ubuntu Linux or WSL2 is sufficient. A modern dual-core processor, at least 4 GB RAM, and approximately 200 MB of free disk space are recommended. Local testing can use the loopback interface; network testing requires connectivity between client and server.

3.2 Software Requirements

- Ubuntu Linux or Ubuntu under WSL2.
- GCC compiler and GNU Make.
- Terminal application for running the server and multiple clients.
- POSIX/Linux headers and libraries provided by the operating system.

3.3 Operating System Environment — Ubuntu / WSL

The project is designed for a POSIX-like Linux environment. On Windows, Ubuntu can be installed through WSL2. Build and run the program inside the Ubuntu terminal.

3.4 Development Tools and Compilers

GCC compiles the C source files, while Make automates the build process using the supplied Makefile. Standard Linux command-line tools are used to navigate directories and execute the programs.

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

4.1 C Programming

C provides direct access to POSIX APIs and is commonly used for operating-system and system-programming exercises.

4.2 GCC Compiler

GCC compiles the client and server source files into executable programs. The supplied Makefile enables a repeatable build.

4.3 Linux System Calls / APIs

- socket() — creates a network endpoint.
- bind() — assigns the server socket a local address and port.
- listen() — prepares a socket to accept connections.
- accept() — accepts a client connection.
- connect() — connects a client to a server.
- fork() — creates a child process.
- pipe() — creates an IPC channel.
- read() and write() — transfer data through file descriptors.
- send() and recv() — exchange network data.
- select() — waits for input on multiple descriptors.
- sigaction() — installs signal handlers.
- waitpid() — collects child-process status.
- close() — releases a file descriptor.
- getpid() — obtains the current process ID.

4.4 Linux Commands and Utilities

The project uses commands such as cd, ls, make, and standard terminal execution. The build command is make, and the server/client are started from the project's bin directory.

4.5 Other Tools Used

Ubuntu/WSL provides the execution environment. GNU Make coordinates compilation. The supplied README and docs directory provide project-specific architecture, system-call, and viva notes.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system contains three main logical components: the client program, the server parent process, and a child process for each connected client. TCP sockets carry network messages. A pipe carries structured events from each child to the parent.

5.2 System Workflow

1. The server creates a TCP socket and binds it to the selected port.
2. The server calls listen() and waits for incoming connections.
3. accept() returns a connected socket for a new client.
4. The server creates an IPC pipe and calls fork().
5. The child handles the connected client and reads client commands.
6. The child sends structured events through the pipe.
7. The parent reads events and updates the active-client list.
8. The parent broadcasts messages or routes private messages through client sockets.
9. When a client disconnects, the parent removes the session and handles process cleanup.

5.3 Flowchart / Data Flow

Client keyboard input → client TCP socket → server child process → IPC pipe → server parent process → selected client TCP socket(s) → recipient terminal.

For submission, insert a flowchart or screenshot of the architecture from your project files if your faculty requires a diagram.

5.4 Module Description

Client module: connects to the server, requests a username, monitors keyboard and socket activity, sends commands, and displays incoming messages.

Server module: accepts connections, creates child processes, receives IPC events, maintains the active-client list, and routes messages.

IPC module: defines structured events used to report login, message, private-message, online-user, and quit events.

Signal and cleanup module: handles shutdown requests and reaps child processes.

5.5 Input and Output Design

Inputs include a server IP address, port, username, and chat commands. Outputs include connection status, chat messages, private messages, the list of online users, help text, and join/leave notifications.

CHAPTER 6 — IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is separated into client/client.c, server/server.c, and common/protocol.h. The documentation and scripts are maintained in their own folders.

6.2 Algorithm / Pseudocode

Server algorithm:

1. Create and configure a TCP listening socket.
2. Install signal handlers.
3. Accept a new client.
4. Create a pipe and fork a child process.
5. Child handles client commands and reports events through the pipe.
6. Parent receives events and routes messages.
7. Remove disconnected clients and reap terminated children.
8. On shutdown, close resources and exit.

Client algorithm:

1. Read server address and port from command-line arguments.
2. Create a socket and connect to the server.
3. Enter a username.
4. Use select() to monitor keyboard input and server messages.
5. Send user input to the server and display received messages.
6. Exit and close the socket when /quit is entered or the connection ends.

6.3 Core C Implementation

The implementation uses standard POSIX socket, process, pipe, signal, and I/O interfaces. The full source code remains in the supplied project archive; this report describes the major functions and design rather than duplicating every source file.

6.4 Linux System Calls / APIs Used

The server relies on socket(), bind(), listen(), accept(), fork(), pipe(), read(), write(), send(), recv(), sigaction(), waitpid(), and close(). The client uses socket(), connect(), select(), send(), recv(), and close().

6.5 Process Implementation

A child process handles each connected client. The parent process continues accepting clients and processing events. This separation illustrates process-based concurrency.

6.6 IPC Implementation

The child writes a structured event to the pipe. The parent reads that event and performs the requested state update or message-routing operation. The parent remains the owner of the client table because ordinary memory changes after fork() are not automatically shared.

6.7 Resource Management

Sockets and pipe descriptors must be closed when they are no longer needed. Signal handling and waitpid() help the server collect child-process exit status and avoid leaving zombie processes.

CHAPTER 7 — EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open the Ubuntu/WSL terminal, extract the project archive, and change into the multi_user_chat directory. Compile the project with:

make

7.2 Execution Commands

Start the server:
./bin/server 8080

Open separate terminals and start clients:
./bin/client 127.0.0.1 8080

For local testing, 127.0.0.1 is the loopback address. For another computer on a trusted LAN, use the server's reachable IP address and allow the selected port through the firewall if required.

7.3 Sample Inputs and Outputs

Supported commands include:
/help
/users
/msg <user> <message>
/broadcast <message>
/quit

Ordinary text is used for a broadcast message. For example, /msg Rahul Hello! sends a private message to a connected user named Rahul, while /broadcast Meeting starts now sends a group message.

7.4 Screenshots of Execution

Insert genuine screenshots from your own running project: successful compilation, server startup, at least three connected clients, broadcast messaging, private messaging, /users output, a client leaving, and server shutdown.

7.5 Demonstration of OS Concepts

During the demonstration, explain how fork() creates a child process, how pipe() carries events to the parent, how TCP sockets transfer messages, how select() allows the client to monitor input sources, and how SIGCHLD/waitpid() support child cleanup.

CHAPTER 8 — CONCLUSION AND RESULTS

8.1 Summary of the Project

The project implements a terminal-based multi-user chat application in C using Linux system calls and POSIX APIs. Multiple clients connect to a server and exchange messages through TCP.

8.2 Objectives Achieved

The architecture demonstrates process creation, IPC, socket communication, command handling, signal processing, and resource cleanup. The source is organized into client, server, and shared protocol components.

8.3 Results and Learning Outcomes

The project provides a practical way to study how an operating system manages processes and file descriptors in a networked program. Actual runtime results should be documented using screenshots and observations from the team's own test run.

CHAPTER 9 — REFERENCES

10. GNU Compiler Collection (GCC) documentation: https://gcc.gnu.org/onlinedocs/
11. GNU Make Manual: https://www.gnu.org/software/make/manual/
12. Linux man-pages: https://man7.org/linux/man-pages/
13. Michael Kerrisk, The Linux Programming Interface, No Starch Press, 2010.
14. W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff, UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd Edition, 2003.
15. Project documentation supplied with the source archive: README.md, docs/ARCHITECTURE.md, docs/SYSTEM_CALLS.md, and docs/VIVA.md.